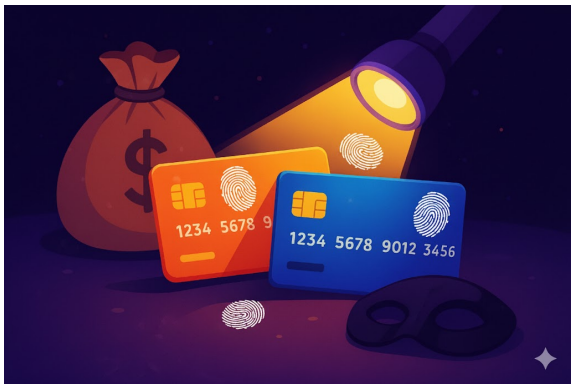
Metodología de la Programación

Curso 2025/2026



Guion de prácticas

Fraud2

Introducción al uso de memoria dinámica. Comparativa entre procesos de clustering

Abril de 2026

Silvia Acid, Andrés Cano, Luis Castillo
Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Granada

Licencia Creative Commons con Reconocimiento - No Comercial - Compartir Igual 4.0 (CC BY-NC-SA 4.0)

Índice

1. Introducción **5**

2. Arquitectura de las prácticas **5**

3. Objetivos **6**

4. Lectura de ficheros. Ficheros loc **7**

5. Vector dinámico con capacidad adicional **9**

6. Práctica a entregar **11**
6.1. Finalidad del programa 11
6.2. Sintaxis y ejemplos de ejecución 12
6.3. Módulos del proyecto 14
6.4. Entrega de la práctica 16

7. Código para la práctica **17**
7.1. Location.h 17
7.2. VectorLocation.h 20
7.3. VectorInt.h 23
7.4. Clustering.h 25
7.5. ArrayClustering.h 29
7.6. main.cpp 30

1. Introducción

En esta práctica conservamos las clases que hemos desarrollado en la práctica anterior y añadiremos el nuevo módulo `ArrayClustering`, que no desarrolla una nueva clase, sino que proporciona un conjunto de funciones externas que operan sobre el tipo de dato del mismo nombre (`ArrayClustering`).

El conjunto de funciones externas que desarrollaremos en el módulo `ArrayClustering` nos servirá para dar nuestros primeros pasos con el uso de memoria dinámica. Este módulo desaparecerá en la práctica Fraud3, pues ya no lo necesitaremos más.

Al igual que en Fraud1, la finalidad del programa de esta práctica es hacer clustering de un conjunto de localizaciones. Sin embargo, ahora realizaremos varios procesos de clustering diferentes. La diferencia entre dos procesos estará en la asignación inicial aleatoria de los clústeres, lo que provocará que probablemente se obtengan distintos conjuntos de clústeres al final de cada proceso.

A partir de esta práctica, vamos a cambiar la forma de ejecución de nuestros programas. Se dejan de realizar lecturas desde teclado (o su equivalencia mediante redireccionamiento). Las lecturas de datos se van a realizar directamente desde ficheros, cuyos nombres, junto con argumentos adicionales, se van a especificar desde la línea de comandos, esto es, a través de los parámetros con los que ejecutamos nuestro programa.

2. Arquitectura de las prácticas

En `Fraud2` conservamos las clases que teníamos en la práctica anterior, que en principio no necesitan de ningún cambio. Solo añadiremos el nuevo módulo `ArrayClustering`. Como siempre, puede ver en la figura 1 la lista de módulos que usaremos.

Describamos brevemente los módulos que usaremos en esta práctica:

A Location
: La clase Location se implementó en prácticas anteriores y no necesita de ningún cambio en esta práctica.

B VectorLocation
: La clase VectorLocation no necesita de ningún cambio en esta práctica.

C VectorInt
: La clase VectorInt no necesita de ningún cambio en esta práctica.

D Clustering
: La clase Clustering no necesita de ningún cambio en esta práctica.

G ArrayClustering
: Nuevo módulo que define el tipo de dato `ArrayClustering` y contiene además un conjunto de funciones externas que operan sobre

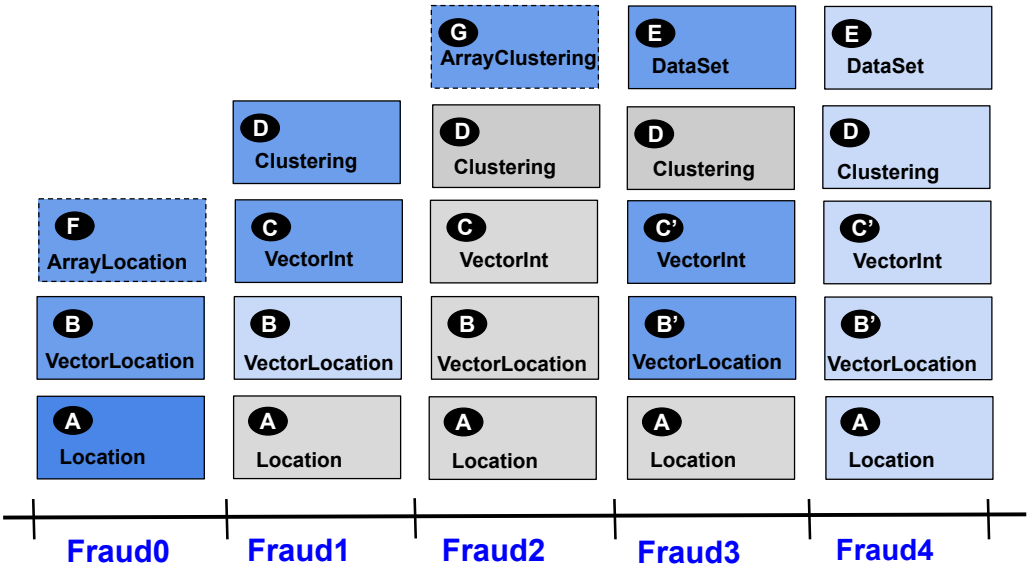


Figura 1: Arquitectura de las prácticas de MP 2026. Como podemos observar, los bloques correspondientes a cada módulo, aparecen en distintos colores. El color azul indica software que requiere ser desarrollado por el estudiante. Los cambios considerables (como los cambios en la estructura interna de una clase) se muestran en color intenso, mientras que pequeños cambios, como la incorporación de nuevas funcionalidades, se muestran en su correspondiente color suave. Finalmente, en gris se muestran módulos que no sufren cambios respecto a la versión anterior de las prácticas.

ese tipo de dato. Como veremos, un objeto `ArrayClustering` sirve para almacenar en memoria dinámica un array de objetos de la clase `Clustering`.

main.cpp Contiene la función main() cuya finalidad será la de obtener un conjunto de agrupamientos, cada uno usando una semilla diferente, para un conjunto de localizaciones.

3. Objetivos

El desarrollo de la práctica `Fraud2` persigue los siguientes objetivos:

- Practicar con el tipo de dato puntero.
- Practicar el paso de objetos `struct` como parámetros de funciones.
- Aprender a utilizar memoria dinámica en C++.
- Trabajar con arrays dinámicos.
- Aprender a leer datos de un fichero de texto mediante el operador >> y la función `getline()`.

- Conocer cómo leer los parámetros pasados a la función `main()` desde la línea de comandos.
- Practicar con el depurador de programas de VSCode y con la herramienta `valgrind` para rastrear errores en tiempo de ejecución.

4. Lectura de ficheros. Ficheros loc

En esta práctica los datos sobre el conjunto de localizaciones, en lugar de leerlos desde la entrada estándar, se van a leer desde ficheros de texto con extensión `.loc`. Un fichero `loc` es un fichero de texto que contiene información acerca de un conjunto de localizaciones. Un fichero `loc` tiene el siguiente formato:

- La primera línea contiene un número entero positivo con el número n de localizaciones que vienen a continuación.
- Las siguientes líneas contienen la información de cada una de las n localizaciones. Cada línea contiene la información de una localización (coordenada x, coordenada y, nombre). Los campos de cada localización están separados con uno o varios caracteres blanco. El campo nombre podría estar compuesto de varias palabras separadas por un espacio en blanco. También es posible que el campo nombre esté vacío. Es posible que aparezcan uno o varios espacios en blanco antes o después del valor de cada campo, de la misma forma que ocurría en Fraud1.

Ejemplo 1 *Ejemplo de fichero loc con información de 5 localizaciones: DataSets/dataP2/simple_5locations.loc*

```
5
0.0 0.0 A
1.0 1.0 B
2.0 2.0 C
3.0 3.0 D
4.0 4.0 E
```

Ejemplo 2 *Ejemplo de fichero loc con información de 21 localizaciones: DataSets/dataP2/princeton_names.loc*

```
21
3.3 9.1 Graduate College
11.8 6.9 Wawa Public Restrooms
12.6 7.3 Princeton Station
10.1 9.7 Forbes College
13.3 12.2 Dinky Bar & Kitchen
15.2 11.2 Whitman College
12.6 17.5 Admission Information Center
12.5 20 Rockefeller College
13.7 22.8 Art Museum Store
17 23.3 Princeton University Store
18.6 22.8 Labyrinth Books
20.9 15.4 Frist Campus Center
23.7 14.8 Cannon Dial Elm
24.8 14.9 Quadrangle
25.6 14.9 Ivy
```

```
26.4 14.9 Cottage
27.3 14.5 Cap & Gown
28.2 15 Cloister
29.1 15.2 Charter
28.8 18.7 Engineering Quadrangle
18.5 6.7 Yeh College
```

Los datos de un fichero de texto pueden procesarse de forma muy similar a como se leen datos desde el teclado (entrada estándar) o como se escriben datos en pantalla (salida estándar) con los operadores de extracción >> y de inserción << en un flujo de datos. En la lectura, también podemos emplear funciones como `getline()` (para leer una línea), `get()` (para leer un carácter).

Los pasos a seguir para proceder con la lectura de datos desde un fichero son los siguientes:

1. Incluir el fichero de cabeceras `fstream` para poder manejar ficheros.
2. Crear un flujo de entrada de datos `ifstream` asociándolo al fichero del que se van a leer los datos.
3. Comprobar si el flujo se abrió correctamente. En caso de que el fichero no exista, no tenga permisos de lectura u otro tipo de situación de error, el flujo no podrá crearse.
4. Leer los datos con el operador de extracción >> exactamente igual que si se leyesen desde `cin`. También podemos usar otras funciones como `getline()` o `get()`.
5. Cerrar el flujo de entrada, una vez termina la lectura de los datos.

En el siguiente código de ejemplo, mostramos un programa que lee el contenido del fichero de localizaciones del ejemplo 1, y lo muestra por la salida estándar.

```
#include <iostream>
#include <fstream>
using namespace std;
int main(int argc, char* argv[]){
  ifstream inputStream;
  int nLocs;
  int x, y;
  string name;

  inputStream.open("simple_5locations.loc"); // Abrimos el flujo
  if(inputStream){ // Comprobamos que el flujo se abrió correctamente
     inputStream >> nLocs; // Leemos el número de localizaciones
     cout << nLocs;
     for(int i=0; i<nLocs; i++){
       inputStream >> x; // Leemos coordenada x
       inputStream >> y; // Leemos coordenada y
       getline(inputStream, name); // // Leemos el nombre
       cout << x << " " << y << " " << name << endl;
     }
  }
  inputStream.close(); // Cerramos el flujo
}
```

En el caso de esta práctica Fraud2, tenga en cuenta que podemos utilizar el método `load()` de la clase `VectorLocation` para leer un fichero de localizaciones. Este método ya lo deberíamos tener implementado de la práctica Fraud1. En Fraud2, el nombre del fichero de localizaciones se proporciona al programa mediante el argumento número 4 en la línea de comandos. De esta forma, la forma de leer el fichero de entrada sería similar al siguiente código:

```
#include <iostream>
#include <fstream>
#include "VectorLocation";

using namespace std;
int main(int argc, char* argv[]){
  ifstream inputStream;
  VectorLocation locations;

  inputStream.open(argv[4]); // Abrimos el flujo
  if(!inputStream){
    showHelp(cerr, "Error opening input file: " + string(argv[4]));
    return 1;
  }
  locations.load(inputStream);
  inputStream.close(); // Cerramos el flujo
}
```

5. Vector dinámico con capacidad adicional

Como se ha visto en las clases de teoría, existen varias formas para gestionar vectores dinámicos.

1. Una de ellas sería hacer que en todo momento, la reserva de memoria dinámica del vector se ajuste al número de objetos que estemos usando en ese momento. Esta forma hace que se necesiten tan solo dos datos para gestionar un vector dinámico: un puntero (llamado por ejemplo `_array`) al primer elemento del vector y un entero que indique el número de elementos de que disponemos (llamado por ejemplo `_util`). En esta situación, cada operación de añadir un objeto supondría realojar el vector [1].

2. Una forma alternativa consiste en llevar por un lado, la cuenta del espacio reservado (o sea, la *capacidad*) del vector y por otro, el número de objetos que estemos usando en cada momento. Llamemos por ejemplo, `_capacidad` y `_util` a los datos que guardan estos valores. Dentro de esta alternativa, a su vez, existen varias opciones en la forma en que el vector dinámico se realoja cuando se agota su capacidad:

   a) Una opción sería hacer que el nuevo vector multiplique su capacidad por algún factor dado respecto a la que tenía anteriormente.

   b) Otra opción sería hacer que el nuevo vector tenga como capacidad la que tenía anteriormente, más un bloque en el que quepan un número fijo de elementos adicionales.

En esta práctica, usaremos la alternativa 2, ya que permite que el vector dinámico no deba ser continuamente realojado conforme le vamos añadiendo nuevos elementos. Y concretamente nos decantamos por la opción 2*b*.

Ejemplo 3 *Sea un vector reservado en un bloque de capacidad para 5 objetos, con tan solo 3 objetos operativos. Un esquema puede verse en la figura 2.*

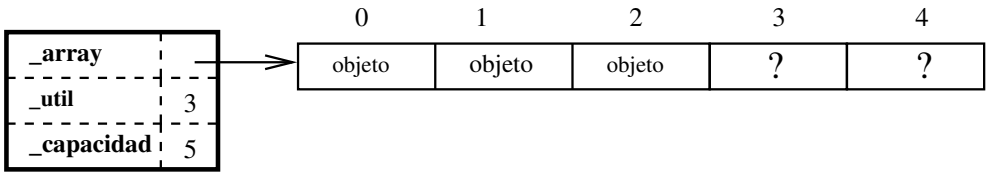


Figura 2: Reserva de espacio para 5 objetos, aunque el vector tan solo disponga de 3 objetos operativos, el resto están a valores por defecto (valor ?).

En el estado de la figura 2, a la llegada sucesiva de dos nuevos objetos, estos se podrán alojar en el mismo vector hasta ocupar la capacidad total, sin tener que realojar el vector. En este caso, basta con incrementar `_util`, como podemos ver en la figura 3.

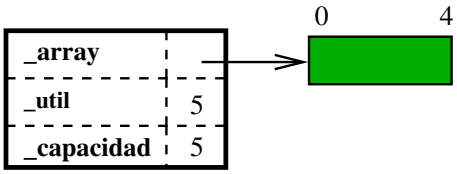


Figura 3: Reserva de espacio para 5 objetos, el vector está completo (`_util` coincide con `_capacidad`).

Siguiendo con nuestro ejemplo, si llegan nuevos objetos, será necesario realojar el vector en otro espacio redimensionado a una capacidad mayor. Supongamos que cada vez que se agota la capacidad del vector, la aumentamos en 5. Al hacerlo, tendríamos un nuevo vector con capacidad para 10 objetos tal como podemos ver en la figura 4 a).

Veamos con detalle, cómo se procede cuando se agota la capacidad del vector dinámico (_util coincide con `_capacidad`) y llega un nuevo objeto para ser insertado en el vector. Será necesario realojar el vector anterior en uno más grande, copiar toda la información previa en el nuevo espacio y finalmente liberar el espacio anterior. Gráficamente el proceso puede seguirse en la figura 4.

Los pasos que se han seguido al insertar un nuevo elemento son los siguientes:

1. En (a), el vector está al completo a la llegada del nuevo objeto marcado en rojo.

2. En b) se reserva nueva memoria dinámica, mediante el puntero auxiliar `temp`, con capacidad para $10 + 5$ objetos.

3. En c) se copia el contenido del vector dinámico en el nuevo vector apuntado por `temp`.

4. En d) se libera la memoria dinámica del vector dinámico original.

[1]Como ejercicio didáctico es muy útil pero, está muy lejos de ser realista, dado el esfuerzo que supone el realojo, que al final se traduce en tiempos.

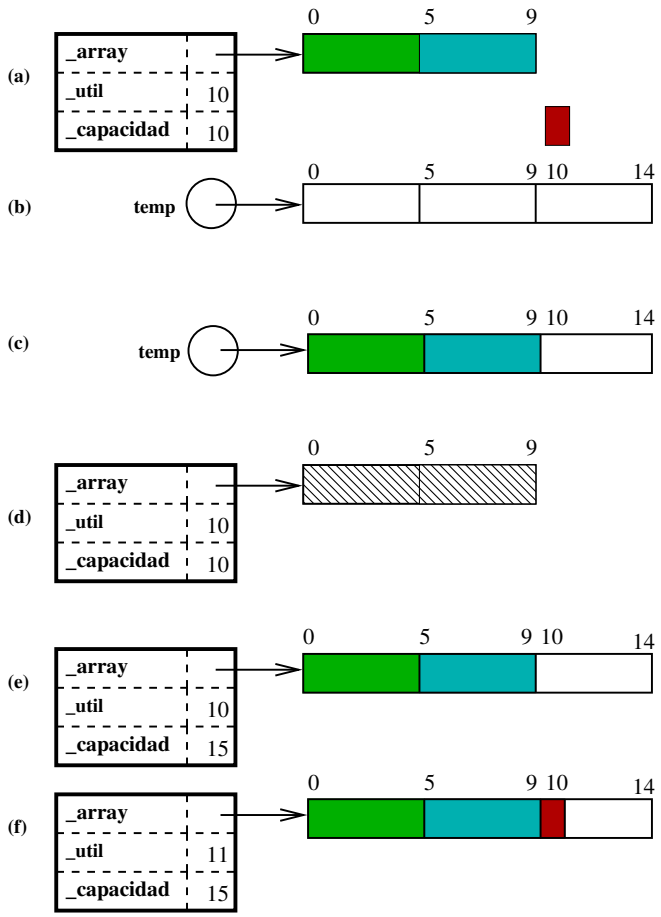


Figura 4: Proceso completo de realojo y adición de un elemento.

5. En e) se corrige la dirección del puntero `_array` para que apunte a la dirección de `temp`, y se ajusta el valor de la capacidad del nuevo vector. Llegado a este punto, ya se ha llevado a cabo el realojo del vector en uno de mayor capacidad.

6. Por último, en f) se añade al vector dinámico el nuevo objeto, ya que hay espacio disponible. Se ajusta también el valor del número de elementos usados en el vector.

Si la gestión del vector dinámico se hubiera hecho con la alternativa 1, habría que haber realojado el vector con cada operación de adición de un objeto. Con la alternativa elegida, se logra reducir el esfuerzo de realojo.

6. Práctica a entregar

6.1. Finalidad del programa

El propósito del programa Fraud2 es leer los datos de un conjunto de localizaciones desde un fichero de texto cuyo nombre se proporciona a través de la línea de comandos y realizar un determinado número de procesos de clustering sobre tal conjunto de localizaciones, mostrando luego varias estadísticas sobre cada clustering según se detalla más adelante.

El programa también recibirá por la línea de comandos (ver más adelante la sintaxis de ejecución del programa) el valor `K` usado para definir el número de clústeres a usar en cada proceso de clustering, y dos valores enteros para definir el intervalo de semillas de números aleatorios.

En cada proceso de clustering usaremos un valor diferente de semilla. Como puede recordar de la práctica Fraud1, la semilla se utiliza en el método `initialClusterAssignment()` de la clase `Clustering` para inicializar el generador de números aleatorios, lo cual influye en la inicialización aleatoria de los clústeres que obtiene este método. El uso de semillas diferentes en cada clustering hace que probablemente el conjunto de clústeres obtenidos sea diferente.

El programa guardará en el array dinámico contenido en el objeto `ArrayClustering` el resultado de cada proceso de clustering. Este array será ordenado, según se comenta más abajo, y se mostrará en la salida estándar.

El programa llevará a cabo los siguientes pasos:

1. Obtener el valor de K y los valores mínimo (min) y máximo (max) que determinan el intervalo de valores de semilla, a partir de los argumentos pasados a la función `main()`.

2. Leer el conjunto de localizaciones desde el fichero cuyo nombre se proporciona al programa como último parámetro de la línea de comandos.

3. Para cada valor de semilla s_i ($s_i \in \mathbb{Z} : min \leq s_i \leq max$):

   a) Realizar el proceso de clustering usando la semilla s_i obteniendo un objeto `Clustering` CL_{s_i}.

   b) Guardar el objeto `Clustering` CL_{s_i} obtenido en el array dinámico, siempre y cuando no sea equivalente a otro objeto ya incluido en el array. Recuerde que dos objetos `Clustering` son equivalentes si contienen los mismos valores K, $sumWCV$ y número de iteraciones.

4. Ordenar el array de objetos `Clustering` en orden ascendente del valor de las sumas de las *within-cluster-variances* ($sumWCV(CL_{s_i})$). Si dos objetos `Clustering` CL_{s_i} y CL_{s_j} contienen el mismo valor ($sumWCV(CL_{s_i}) = sumWCV(CL_{s_j})$), los ordenaremos entre ellos de forma ascendente por el número de iteraciones que necesitó cada uno.

5. Para cada objeto `Clustering` CL_{s_i} del array, mostrar sus estadísticas (valor K, $sumWCV(CL_{s_i})$ y número de iteraciones realizadas).

6.2. Sintaxis y ejemplos de ejecución

Como se indicó en la Introducción, en esta práctica cambiamos la forma de ejecutar nuestros programas respecto a prácticas anteriores. En

esta práctica usaremos los parámetros de la línea de comandos, para pasarle al programa los datos que necesita para su ejecución. En Fraud2, el programa recibirá cuatro parámetros por la línea de comandos.

La **sintaxis de ejecución** del programa desde un terminal es:

```
Fraud2> build/Fraud2 <K> <minSeed> <maxSeed> <inputFile.loc>
```

Veamos algunos ejemplos de ejecución del programa desde la línea de comandos:

Ejemplo 4 *Faltan argumentos para la ejecución del programa:*

```
Fraud2> build/Fraud2 5 1 3
```

La salida del programa será la que muestra la función `showHelp()`*, incluida en* `main.cpp`*, usando el mensaje* `"Not enough arguments"`*:*

```
ERROR in Fraud2 parameters. Not enough arguments
Run with the following arguments:
Fraud2 <K> <minSeed> <maxSeed> <inputFile.loc>

Parameters:
<K>: an integer with the number of clusters to use in the K-means algorithm
<minSeed>: an integer with the minimum seed value to use
<maxSeed>: an integer with the maximum seed value to use
<inputFile.loc>: name of the location file
```

Ejemplo 5 *Clustering del conjunto de localizaciones contenidas en el fichero* `princeton_names.loc` *(ver ejemplo 2), usando* $K = 5$ *y el intervalo de semillas* $s \in [0, 3]$*:*

```
Fraud2> build/Fraud2 5 0 3 ../Datasets/dataP2/princeton_names.loc
```

La salida que debería obtenerse al ejecutar el programa sería la siguiente:

```
Clustering 0:
K=5
Sum of within-cluster variances: 194.455556
Number of iterations: 3

Clustering 1:
K=5
Sum of within-cluster variances: 233.319889
Number of iterations: 3

Clustering 2:
K=5
Sum of within-cluster variances: 250.044571
Number of iterations: 3
```

Fíjese que la salida del programa muestra datos sobre tres procesos de clustering y no de cuatro como podría esperarse. El cuarto objeto clustering no fue incluido en el array de objetos `Clustering` *porque era equivalente (mismo valor para los valores:* K*,* $sumWCV$ *y número de iteraciones) a otro de los obtenidos anteriormente. Esto está especificado así en los comentarios de la función* `AppendArrayClustering()`*, como puede ver en la sección 7.5.*

La salida del programa mostrada más arriba puede verse también en el fichero `CodeProjects/Datasets/dataP2/fraud2_test09.p2out`*.*

En la carpeta `CodeProjects/Datasets/dataP2`, puede encontrar varios ficheros de localización con extensión `.loc` con los que podrá hacer diferentes pruebas.

Además, se proporcionan una serie de ficheros con extensión `.test` que contienen tests de integración, y que pueden usarse con el script `runTest.sh` [2]. Veamos un ejemplo de ejecución de este script:

Ejemplo 6 *Ejecución de* `runTest.sh` *con un fichero de tests:*

```
Fraud2> scripts/runTest.sh tests/fraud2_test09.test
```

La salida que debería obtenerse al ejecutar el anterior script sería la siguiente:

```
Loading ansiterminal...

VALIDATION TOOL v2.0
Checking tests folder           OK
Setting everything up           OK
Test #1  interval for three clustering. K=5. Three registered clusterings

Testing tests/fraud2_test09.test                Generating fresh binaries       --
↪  Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to:
↪  /home/gte/acu/Latex/Docencia/MP/MP_2025_2026/Github/CodeProjects/Fraud2Teacher/build

[  OK  ] Test #1 [tests/fraud2_test09.test] (valgrind --leak-check=full
↪  build/Fraud2Teacher  5 0 3 ../Datasets/dataP2/princeton_names.loc)
         ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
End of tests
```

El contenido del fichero de test utilizado (`fraud2_test09.test`*) es el siguiente:*

```
%%%CALL 5 0 3 ../Datasets/dataP2/princeton_names.loc
%%%VALGRIND
%%%DESCRIPTION interval for three clustering. K=5. Three registered clusterings
%%%OUTPUT
Clustering 0:
K=5
Sum of within-cluster variances: 194.455556
Number of iterations: 3

Clustering 1:
K=5
Sum of within-cluster variances: 233.319889
Number of iterations: 3

Clustering 2:
K=5
Sum of within-cluster variances: 250.044571
Number of iterations: 3
```

Como puede verse, la salida que aparece tras la línea `%%%OUTPUT` *es la que se mostraba también en el ejemplo 5 como salida del programa.*

6.3. Módulos del proyecto

A la hora de implementar los nuevos métodos y funciones de esta práctica, fíjese en las cabeceras de tales métodos y funciones en los ficheros `*.h` correspondientes y en los comentarios que les acompañan,

[2] Los detalles de como ejecutar tests de integración de forma automática se encuentran en el guion Fraud0.

pues en ellos están detalladas sus especificaciones. Concretamente, debe revisar las funciones que aparecen en el módulo `ArrayClustering`.

Nota importante: Se han retirado a propósito todos los `const` y `&` para los parámetros de las nuevas funciones externas. El número de argumentos y los tipos han sido establecidos y **no se han de cambiar**.

Se le invita a definir todos los métodos y funciones externas adicionales que estime oportuno para una adecuada modularización del código.

En esta práctica debe tener en cuenta las especificaciones indicadas en los siguientes módulos:

- Clase `Location`. En esta clase no se necesita ningún cambio respecto a la práctica anterior.
- Clase `VectorLocation`. En esta clase no se necesita ningún cambio respecto a la práctica anterior.
- Clase `VectorInt`. En esta clase no se necesita ningún cambio respecto a la práctica anterior.
- Clase `Clustering`. En esta clase no se necesita ningún cambio respecto a la práctica anterior.
- Módulo `ArrayClustering`. El fichero **ArrayClustering.h** (ver sección 7.5) contiene la declaración del tipo `ArrayClustering`, que utiliza un `struct` de C++.

```
const int INITIAL_ARRAY_CLUSTERING_CAPACITY = 2;
const int ARRAY_CLUSTERING_CAPACITY_INCREMENT = 2;

struct ArrayClustering{
    Clustering *clustering; // Pointer to the dynamic array of Clustering objects
    int capacity; // Capacity of the dynamic array
    int size; // Current number of Clustering objects stored in the array
};
```

  Un objeto `ArrayClustering` representa un array dinámico de objetos `Clustering`. Cada objeto del array habrá sido obtenido con el uso de una semilla diferente en el proceso de clustering, lo cual provoca que el conjunto de clústeres obtenidos sea diferente para cada objeto del array.

  El array dinámico anterior se gestionará en la forma que se ha explicado en la sección 5. O sea, el array será creado con una determinada capacidad inicial. A continuación podremos añadirle los objetos que queramos. Cuando la capacidad se agote, el array dinámico será realojado con una capacidad adicional. Como puede ver en la definición anterior del tipo `ArrayClustering`, el dato `clustering` es el puntero al array dinámico, el dato `capacity` nos da la capacidad actual del array y `size` el número de objetos que contiene el array actualmente. Las dos constantes enteras nos indican respectivamente la capacidad inicial que debe tener el array dinámico de un objeto `ArrayClustering` y el tamaño de bloque adicional

que usaremos cuando se necesite aumentar la capacidad del array dinámico [3].

El fichero **ArrayClustering.h** contiene además el prototipo de varias funciones externas, que nos permiten gestionar los objetos `Array Clustering`. Recuerde que debe revisar y/o modificar todas las cabeceras de estas funciones.

```
void InitializeArrayClustering(ArrayClustering arrayClustering,
    int capacity=INITIAL_ARRAY_CLUSTERING_CAPACITY);
void DeallocateArrayClustering(ArrayClustering arrayClustering);
int FindArrayClustering(ArrayClustering arrayClustering,
                        Clustering clustering);
void AppendArrayClustering(ArrayClustering arrayClustering,
                           Clustering clustering);
void SortArrayClustering(ArrayClustering arrayClustering);
```

- Módulo `main.cpp`. Este programa tiene por finalidad lo descrito en la sección 6.1. Teniendo en cuenta los detalles comentados en tal sección y los pasos esbozados en el código proporcionado para esta función (sección 7.6), complete el código de este módulo.

Recuerde además corregir los fallos detectados en la práctica anterior, pues se procede con una nueva evaluación de cada uno de los métodos implementados, por lo que los errores no corregidos volverían a penalizar en esta práctica.

6.4. Entrega de la práctica

La práctica deberá ser entregada en Prado, durante el periodo habilitado en la propia actividad de entrega, y consistirá en un fichero ZIP del proyecto. El nombre del fichero puede ser cualquiera, pero tendrá la extensión zip. Por ejemplo, podría llamarse `PRACTICAMP.zip`. Para obtener este fichero se sugiere utilizar el script `runZipProject.sh` (ver el guion de `Fraud0`). Compruebe que lo entregado es compilable y operativo y, no olvide poner el nombre de los componentes del equipo en la cabecera del fichero `main.cpp`.

[3] Por razones didácticas, se han elegido unos valores bajos para estas constantes, pero debemos decir que estos valores están lejos de los tamaños de bloque adecuados.

7. Código para la práctica

7.1. Location.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file Location.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 6 de octubre de 2025, 12:27
 */

#ifndef LOCATION_H
#define LOCATION_H

#include <iostream>
#include <string>

/**
 * @class Location
 * @brief It stores data about a location where people spend money on some
 * service or product. A location contains data about the x and y position
 * (double values) in the 2D space and the name of the location
 * (a string that can contain several words).
 */
class Location {
public:
    /**
     * @brief Constructor that builds a Location object with a value of 0
     * for the x and y coordinates, and an empty string for the name.
     */
    Location();

    /**
     * @brief Returns the x-coordinate of this location
     * Query method
     * @return The x-coordinate of this location
     */
    double getX() const;

    /**
     * @brief Returns the y-coordinate of this location
     * Query method
     * @return The y-coordinate of this location
     */
    double getY() const;

    /**
     * @brief Returns the name of this location
     * Query method
     * @return The name of this location
     */
    std::string getName() const;

    /**
     * @brief Checks if the position of this object is inside the area
     * determined by the two given locations: Location A (@p bottomLeft) and
     * B (@p topRight).
     *
     *      +------------+(B)
     *      |            |
     *      |            |  +(O3)
     *      |            |
     *      |     +(O2)  |
     *      |            |
     *      |            |
     *   (A)+------------+
     *          +(O1)
     *
     * Examples of returning false:
     * - Object O3 is outside the given area (O3.x > B.x)
     * - Object O1 is outside the given area (O1.y < A.y)
     * Example of returning true:
     * - Object O2 is inside the given area
     * - Object A is inside the given area
     * - Object B is inside the given area
     *
     * Query method.
     * @param bottomLeft The Location of the bottom left point. Input parameter
     * @param topRight The Location of the top right point. Input parameter
     * @return true if the Location of this object is inside the area
     * determined by the two given Locations
     */
    bool isInsideArea(const Location &bottomLeft, const Location &topRight) const;

    /**
     * @brief Calculates the square of the Euclidean distante from this location
     * to the provided location. That is, it loc1 and loc2 are two
     * Location objects, then this method returns:
     * \f$ (loc1.x - loc2.x)^2 + (loc1.y - loc2.y)^2 \f$
```

```
     * Query method
     * @param location A Location object. Input parameter
     * @return Returns the square of the Euclidean distante from this
     * location to the provided location.
     */
    double squaredDistance(const Location& location) const;

    /**
     * @brief Calculates the Euclidean distante from this location to the provided
     * location. That is, it loc1 and loc2 are two Location objects, then
     * this method returns:
     * \f$ \sqrt{ (loc1.x - loc2.x)^2 + (loc1.y - loc2.y)^2 } \f$
     * Note that this method can be implemented using the squaredDistance()
     * method.
     * Query method
     * @param location A Location object. Input parameter
     * @return The Euclidean distante from this location to the provided
     * location.
     */
    double distance(const Location& location) const;

    /**
     * @brief Obtains a string with the x and y coordinates
     * and the name (whitespace separated). If the name is an empty string,
     * then only the x and y coordinates are included in the resulting string.
     * To convert the x and y coordinates to a string you must use the
     * std::to_string(int) C++ function.
     * Query method
     * @return string with information about this Location object
     */
    std::string toString() const;

    /**
     * @brief Sets the x-coordinate in this object
     * Modifier method
     * @param x The new value for the x-coordinate. Input parameter
     */
    void setX(double x);

    /**
     * @brief Sets the y-coordinate in this object
     * Modifier method
     * @param y The new value for the y-coordinate. Input parameter
     */
    void setY(double y);

    /**
     * @brief Sets the name in this object
     * Modifier method
     * @param name A string with the new value for the name. Input parameter
     */
    void setName(const std::string& name);

    /**
     * @brief Sets the x and y coordinates, and the the name in this object
     * Modifier method
     * @param x The new value for the x-coordinate. Input parameter
     * @param y The new value for the y-coordinate. Input parameter
     * @param name A string with the new value for the name. Input parameter
     */
    void set(double x, double y, const std::string& name);

    /**
     * @brief Reads from the provided input stream the information
     * to fill this Location object.
     * @note This method reads two double values (space separated) from the
     * input stream that are used to set the x and y coordinates of this object.
     * Then, it reads the string that can be formed from the current position of
     * the input stream to the end of the line. This string is then trimmed
     * (see the Trim(string) function) to remove its spaces at the beginning
     * and at the end. The resulting string is used to set the name of this
     * Location object. Note that the name can have several words separated by
     * whitespaces.
     * @param is Input stream. Input/output parameter
     */
    void load(std::istream& is);

private:
    /**
     * The x-coordinate of this location
     */
    double _x;

    /**
     * The y-coordinate of this location
     */
    double _y;

    /**
     * The name of this location. It can contain several words.
     */
    std::string _name;
}; // end of class Location

/**
 * Removes spaces and \t characters at the beginning and at the end of the
 * provided string @p myString. If the provided string @p myString is empty
 * or contains only spaces or \t characters then @p myString will contain an
 * empty string after calling to this function.
 * @note This function can be easily implemented using the methods
```

```
 * find_first_not_of(string) and find_last_not_of(string) of class string.
 * @param myString a string. Input/Output parameter
 */
void Trim(std::string & myString);

#endif /* LOCATION_H */
```

7.2. VectorLocation.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file VectorLocation.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 11 de diciembre de 2025, 11:27
 */

#ifndef VECTORLOCATION_H
#define VECTORLOCATION_H

#include "Location.h"

/**
 * @class VectorLocation
 * @brief A VectorLocation object contains a vector of Location objects. It has
 * a capacity (maximun number of Locations that can be stored in the vector)
 * and a size (number of Locations the vector currently contains).
 * The public methods of this class do not allow a VectorLocation to contain two
 * Location objects with identical names.
 * This class uses an array of Location objects with a fixed capacity
 * to store the set of locations.
 */
class VectorLocation {
public:
    /**
     * @brief It builds a VectorLocation object (vector of Location objects)
     * with a size equal to the provided value (@p size).
     * Each element in the vector is initialized with the default Location
     * constructor.
     * @throw std::out_of_range Throws a std::out_of_range exception if
     * @p size < 0 or size>DIM_VECTOR_LOCATIONS
     * @param size The size for the vector of Location in this object. Input
     * parameter
     */
    VectorLocation(int size = 0);

    /**
     * @brief Gets the number of elements in the vector of this object
     * Query method
     * @return The number of elements
     */
    int getSize() const;

    /**
     * @brief Gets the capacity of the vector in this object
     * Query method
     * @return The capacity of the vector in this object
     */
    int getCapacity() const;

   /**
     * @brief Obtains a string with information about this VectorLocation object,
     * in the following format:
     * - First line, the number of Location objects in this vector, converted to
     * a string (using the to_string(int) C++ function).
     * - For each Location, a line with its information, converted to
     * a string with the Location::toString() method.
     *
     * For example, the following is the content of the string for a
     * VectorLocation object with 4 locations:
4
24.8 14.9 Quadrangle
25.6 14.9 Ivy
26.4 14.9 Cottage
27.3 14.5 Cap & Gown
     * Query method
     * @return string with information about this VectorLocation object
     */
    std::string toString() const;

    /**
     * @brief Searches the provided Location in the array of locations in this
     * object. If found, it returns the position where it was found. If not,
     * it returns -1. We consider that position 0 is the first location in the
     * list of locations and this->getSize()-1 the last location.
     * In order to find a location consider only equality in the name field.
     * Query method
     * @param location A Location. Input parameter
     * @return If found, it returns the position where the location
     * was found. Otherwise it returns -1
     */
    int findLocation(const Location &location) const;

    /**
     * @brief Returns a VectorLocation object with those locations whose
     * positions are inside the area determined by the two given Locations.
     * Query method
     * @param bottomLeft The Location of the bottom left point. Input parameter
     * @param topRight The Location of the top right point. Input parameter
     * @return A VectorLocation with the selected Locations.
```

```
 */
VectorLocation select(const Location &bottomLeft,
                      const Location &topRight) const;

/**
 * @brief Removes all the elements in this object, leaving the container
 * with a size equal to 0. It only needs to set the number of elements
 * (_size field) to zero.
 * Modifier method
 */
void clear();

/**
 * @brief Gets a const reference to the Location element at the given
 * position
 * Query method
 * @throw std::out_of_range Throws an std::out_of_range exception if the
 * given position is not valid.
 * @param pos position in the VectorLocation object. Input parameter
 * @return A const reference to the Location element at the given position
 */
const Location &at(int pos) const;

/**
 * @brief Gets a reference to the Location element at the given position.
 * Modifier method
 * @throw std::out_of_range Throws an std::out_of_range exception if the
 * given position is not valid
 * @param pos position in the VectorLocation object. Input parameter
 * @return A reference to the Location element at the given position.
 */
Location &at(int pos);

/**
 * @brief Appends a copy of the given Location object at the first free
 * position in the array of Location in this object. The location is only
 * appended to this object if it was not already found in this object or
 * its name is an empty string.
 * @throw std::out_of_range Throws a std::out_of_range exception if the
 * provided location is going to be appended but the array of Location
 * was full (its capacity was full). If the provided location is not going
 * to be appended because it was already found in this object or its name
 * is an empty string, then no exception is thrown.
 * Modifier method
 * @param value the new Location object to be appended. Input parameter
 * @return true if the given Location could be inserted in this
 * VectorLocation object; false otherwise (the location was already found
 * in this object)
 */
bool append(const Location &location);

/**
 * @brief Appends to this VectorLocation object, the list of
 * Location objects contained in the provided VectorLocation object
 * that are not found (using VectorLocation::findLocation(Location)) in
 * this object.
 * This method could be implemented with the help of the method
 * VectorLocation::append(const Location & location), to append to this
 * object, the Locations of the provided VectorLocation object.
 * Modifier method
 * @param crimeSet A VectorLocation object. Input parameter
 */
void join(const VectorLocation &locations);

/**
 * Sorts the array of locations in this object by increasing alphabetical
 * order of the name of its location (a string).
 * Modifier method
 */
void sort();

/**
 * @brief Gets the position in this vector of the Location object nearest to
 * the provided location
 * Query method
 * @param location A Location object. Input parameter
 * @return the position of the Location object nearest to the provided
 * location.
 * If returns -1 if this vector is empty
 */
int nearest(Location location);

/**
 * Assigns the provided value to all the elements in this vector
 * Modifier method
 * @param location A Location object. Input parameter
 */
void assign(Location location);

/**
 * @brief Reads from the provided input stream the information
 * to fill this VectorLocation object.
 * @note This method should remove any Location previously contained in the
 * provided VectorLocation object. See files *.loc in the folder DataSets
 * as examples of this kind of file.
 * @note This operator throws an exception in some error cases (see below).
 * Before throwing the corresponding exception, this method clears
 * the object (it calls to clear() method) to leave the object in a
 * consistent state.
 * Modifier method
```

```
     * @throw std::out_of_range Throws a std::out_of_range exception if the
     * number of Location read from the input stream is negative.
     * @throw std::out_of_range Throws a std::out_of_range exception if the
     * number of locations read from the input stream is greater than the capacity
     * of this VectorLocation object.
     * @param is Input stream. Input/output parameter
     */
    void load(std::istream is);

private:
    /**
     * Constant with the capacity of the array _locations
     */
    static const int DIM_VECTOR_LOCATIONS = 100;

    /**
     * Array of Locations
     */
    Location _locations[DIM_VECTOR_LOCATIONS];

    /**
     * Number of Location objects contained in the array _locations
     */
    int _size;
}; // end of class VectorLocation

#endif /* VECTORLOCATION_H */
```

7.3. VectorInt.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file VectorInt.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 30 de julio de 2025, 9:54
 */

#ifndef VECTORINT_H
#define VECTORINT_H

#include <iostream>
#include <string>

/**
 * @class VectorInt
 * @brief A VectorInt object contains a vector of integer elements. It has
 * a capacity (maximun number of elements that can be stored in the vector)
 * and a size (number of elements the vector currently contains).
 */
class VectorInt {
public:

    /**
     * @brief It builds a VectorInt object (vector of integers) with a
     * size and capacity equal to the provided value (@p size). Each element
     * will be filled with a value equal to 0.
     * @throw std::out_of_range Throws a std::out_of_range exception if
     * @p size < 0 or size>DIM_VECTOR_VALUES
     * @param size The size for the vector of integers in this object. Input
     * parameter
     */
    VectorInt(int size=0);

    /**
     * @brief Gets the number of elements in the vector of this object
     * Query method
     * @return The number of elements
     */
    int getSize();

    /**
     * @brief Gets the capacity of the vector in this object
     * Query method
     * @return The capacity of the vector in this object
     */
    int getCapacity();

    /**
     * @brief Compares the integer vectors of this object and the provided
     * object, and returns the number of identical elements in both of them. The
     * comparison is performed in order, element by element, in both vectors.
     * For example, given the following two vectors:
     * 2 1 2 3 5
     * 1 1 2 4 5
     * this method will return 3 (there is a match in positions 1, 2 and 4)
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if the sizes of this and the provided object are different
     * Query method
     * @param other A VectorInt object. Input parameter
     * @return The number of identical elements in the vectors of this
     * and the provided object.
     */
    int countIdenticalElements(VectorInt other);

    /**
     * @brief Obtains a string with information about this VectorInt object,
     * in the following format:
     * - First line, the number of elements in this vector.
     * - Second line, the elements in this vector, separated by a whitespace.
     * Take into account that a '\n' should appear after the last element
     * instead of a whitespace.
     * Query method
     * @return string with information about this VectorInt object
     */
    std::string toString();

    /**
     * @brief Gets the Euclidean distance between this and the provided object.
     * The Euclidean distance between two points \f$ P=(p_1, p_2, ... , p_n) \f$
     * and \f$ Q=(q_1, q_2, ... , q_n) \f$ in an n-dimensional space \f$ R^n \f$
     * is the length of the line segment connecting them, calculated as the
     * square root of the sum of the squared differences of their components:
     * \f$ d(P,Q)=\sqrt{ \sum_{i=1}^{n} (p_i-q_i)^{2} } \f$
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if the size of this object and the provided one are not equal
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if the size of the provided object is zero
     * Query method
     * @param other A VectorInt. Input parameter
     * @return The Euclidean distance between this and the provided objects
```

```
     */
    double distance(VectorInt other);

    /**
     * @brief Assigns the provided value to all the elements in this vector
     * Modifier method
     * @param value An integer value. Input parameter
     */
    void assign(int value=0);

    /**
     * @brief Appends the given integer value at the end (first free position)
     * of the array of integers in this object.
     * @throw std::out_of_range Throws a std::out_of_range exception if the
     * array of Location was full (its capacity was full).
     * Modifier method
     * @param value the new integer value to be appended. Input parameter
     */
    void append(int value);

    /**
     * @brief Removes all the elements in this object, leaving the container
     * with a size equal to 0. It only need to set the number of elements
     * (_size field) to zero.
     * Modifier method
     */
    void clear();

    /**
     * @brief Gets a const reference to the integer element at the given
     * position
     * Query method
     * @param pos position in the VectorInt object. Input parameter
     * @throw std::out_of_range Throws an std::out_of_range exception if the
     * given position is not valid.
     * @return A const reference to the integer element at the given position
     */
    int at(int pos);

    /**
     * @brief Gets a reference to the integer element at the given position.
     * Modifier method
     * @param pos position in the VectorInt object. Input parameter
     * @throw std::out_of_range Throws an std::out_of_range exception if the
     * given position is not valid
     * @return A reference to the integer element at the given position.
     */
    int at(int pos);

private:
    /**
     * Constant with the capacity of the array _values
     */
    static const int DIM_VECTOR_VALUES = 100;

    /**
     * Array of integers with the integer elements in this object
     */
    int _values[DIM_VECTOR_VALUES];

    /**
     * Number of elements contained in the array _values
     */
    int _size;

}; // end of class VectorInt

#endif /* VECTORINT_H */
```

7.4. Clustering.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file Clustering.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 22 de octubre de 2025, 11:57
 */

#ifndef CLUSTERING_H
#define CLUSTERING_H

#include "VectorInt.h"
#include "VectorLocation.h"

/**
 * @class Clustering
 * @brief This class is used to obtain a clustering of a given set of locations
 * (Location objects). A clustering algorithm groups the input locations
 * into K disjoint groups (clusters). The clustering algorithm used in this
 * class is the K-Means algorithm. Each group will consist of a disjunct subset
 * of the original location set and a representative location (centroid), which
 * is calculated as the vector of the means of the locations in the group. The
 * run() method in this class executes the algorithm. An example of use is the
 * following:
VectorLocation locations;
// Suppose here code to fill the vector of locations
Clustering clustering;
clustering.set(locations, 5);
clustering.run();
 */
class Clustering {
public:
    /**
     * @brief Constructor that builds a Clustering object initializing
     * the field (_K) for the number of clusters with 0; the field (_isDone)
     * that tells whether the run() method has already been executed
     * with false; the fields _sumWCV and _numIterations with 0 and the field
     * _seed with DEFAULT_RANDOM_SEED.
     * The fields (_locations, _clusters, _centroids) are initialized with
     * the default constructor of their classes.
     */
    Clustering();

    /**
     * @brief Gets the number of clusters
     * Query method
     * @return The number of clusters
     */
    int getK();

    /**
     * @brief Gets the current vector of centroids
     * Query method
     * @return A const reference to the vector of centroids (a VectorLocation
     * object).
     */
    VectorLocation getCentroids();

    /**
     * @brief Indicates whether the clustering algorithm (run() method) has
     * already been executed for this Clustering object.
     * Query method
     * @return true if the run() method has already been executed for this
     * Clustering object; false otherwise
     */
    bool isDone();

    /**
     * @brief Gets the number of locations in the set of locations of this
     * Clustering object.
     * Query method
     * @return The number of locations in the set of locations of this
     * Clustering
     */
    int getNumLocations();

    /**
     * @brief Gets the cluster number for the Location determined by the
     * provided integer (@p locationIndex)
     * Query method
     * @param locationIndex An integer that determines the position of a
     * location in the vector of Location objects. Input parameter
     * @return The cluster number for the Location determined by
     * the provided integer (@p locationIndex). If the
     * clustering algorithm has not been run (_isDone is false), then it returns
     * -1.
     */
    int clusterOf(int locationIndex);

    /**
     * @brief Gets the value of the sum of the within-cluster variances of this
```

```
     * Clustering. This value is obtained from the field _sumWCV
     * @note The returned value will likely be a garbage value if the run()
     * method has not yet been executed (_isDone is false)
     * Query method
     * @return A double with the sum of within-cluster variances of this
     * Clustering
     */
    double getSumWCV();

    /**
     * @brief  Gets the number of iterations used in the KMeans
     * algorithm until no cluster assignment change occurs.
     * It obtains the number of iterations (field _numberIterations) used
     * in the KMeans algorithm until no cluster change occurred.
     * @note The returned value will likely be a garbage value if the run()
     * method has not yet been executed (_isDone is false)
     * Query method
     * @return The number of iterations used in the KMeans algorithm
     */
    int getNumIterations();

    /**
     * @brief  Gets a string with information about the provided cluster (index
     * of a cluster). That string contains a line for each location in the given
     * cluster. The information of each location consists of an integer with
     * its location index (whithin the _location field), a whitespace, and
     * the information returned by the Location::toString() method for that
     * location. An example is the following (a cluster with three locations):
0 3.300000 9.100000 Graduate College
6 12.600000 17.500000 Admission Information Center
7 12.500000 20.000000 Rockefeller College
     * Query method
     * @param cluster An integer, that defines the index of a cluster. Input
     * parameter
     * @return Gets a string with information about the provided cluster. If the
     * clustering algorithm has not been run (_isDone is false) or an
     * invalid value of cluster is provide, then it returns an empty string.
     */
    std::string clusterInfo(int cluster);

    /**
     * @brief Obtains a string with the some statistics data about this
     * Clustering object.
     * The first line contains the value of K; the second line contains the
     * value of the sum of within-cluster variances; the third line contains
     * the number of iterations used in the KMeans algorithm.
     * Query method
     * @return A string with the content described above
     */
    std::string getStatistics();

    /**
     * @brief Indicates whether this Clustering object is equivalent to the
     * provided Clustering object (@p other). Two Clustering objects are
     * considered equivalent if they have the same K value,
     * the same sum of within-cluster variances and the same number of
     * iterations in the KMeans algorithm.
     * If either this Clustering object or the provided Clustering object
     * (@p other) has not yet run the clustering algorithm (the run() method),
     * then they are not considered equivalent.
     * Query method
     * @param other A Clustering object. Input parameter
     * @return true if this Clustering object is equivalent to the provided
     * Clustering object (@p other); false otherwise
     */
    bool isEquivalentTo(Clustering other);

    /**
     * @brief Obtains a string with the fields of this Clustering object.
     * The first line contains the value of K; the second line contains the
     * value of the sum of within-cluster variances; the third line contains
     * the number of iterations used in the KMeans algorithm.
     * In the next three lines, information about the cluster number for each
     * location.
     * If follows information about each centroid
     * Next, for each cluster, information about its locations
     * Here, an example is shown:
K=5
Sum of within-cluster variances: 161.136889
Number of iterations: 5
Cluster number for each location:
21
0 2 2 2 2 2 1 1 1 1 1 4 4 4 4 4 4 4 4 4 3
Centroids:
5
3.300000 9.100000
14.880000 21.280000
12.600000 9.460000
18.500000 6.700000
26.088889 15.366667
Cluster 0 information:
0 3.300000 9.100000 Graduate College
Cluster 1 information:
6 12.600000 17.500000 Admission Information Center
7 12.500000 20.000000 Rockefeller College
8 13.700000 22.800000 Art Museum Store
9 17.000000 23.300000 Princeton University Store
10 18.600000 22.800000 Labyrinth Books
Cluster 2 information:
1 11.800000 6.900000 Wawa Public Restrooms
```

```
2 12.600000 7.300000 Princeton Station
3 10.100000 9.700000 Forbes College
4 13.300000 12.200000 Dinky Bar & Kitchen
5 15.200000 11.200000 Whitman College
Cluster 3 information:
20 18.500000 6.700000 Yeh College
Cluster 4 information:
11 20.900000 15.400000 Frist Campus Center
12 23.700000 14.800000 Cannon Dial Elm
13 24.800000 14.900000 Quadrangle
14 25.600000 14.900000 Ivy
15 26.400000 14.900000 Cottage
16 27.300000 14.500000 Cap & Gown
17 28.200000 15.000000 Cloister
18 29.100000 15.200000 Charter
19 28.800000 18.700000 Engineering Quadrangle
     * Query method
     * @return A string with the content described above
     */
    std::string toString();

    /**
     * @brief Sets the vector of locations (_locations), the value of K (_K)
     * and the seed (_seed) with the provided values.
     * The fields _isDone, _clusters and _centroids_ are also initialized
     * appropriately: _isDone is set to false; _clusters is initialized with
     * a VectorInt object with size equal to the number of locations;
     * _centroids is initialized with a VectorLocation object with size K;
     * the fields _sumWCV and _numIterations are set to 0.
     * Modifier method
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if K<=0
     * @param locations The vector of locations to used in the clustering
     * algorithm. Input parameter
     * @param K The parameter K for the K-Means algorithm. Input parameter
     * @param seed The seed used to initialize the random number generator
     * (with srand(seed)) in the initialClusterAssignment() method.
     * Input parameter
     */
    void set(VectorLocation locations, int K,
        unsigned int seed=DEFAULT_RANDOM_SEED);

    /**
     * @brief Run the clustering algorithm. The clustering algorithm implemented
     * in this method is the KMeans algorithm.
     * After running this method the following postcondition should be verified:
     * @post The _centroids field should contain the vector of the means of
     * locations in each group
     * @post _clusters (a VectorInt object) should contain the cluster number
     * for each location in the vector of locations (_locations)
     * @post The _isDone field should be set to true
     * @post The _sumWCV field should be set to the sum of the within-cluster
     * variances using the claculateSumWithinClusterVariances() method.
     * @post The _numberIterations field should be set to the number of
     * iterations used by the KMeans algorithm.
     *
     * A brief description of the KMeans algorithm is the following:
     *
     * -# Perform an initial assignment of a cluster to each of the Location
     * objects. This must be done by initializing the random number
     * generator using srand(seed);
     * Then, for each location i, assign cluster number rand() % _K
     * to that location. Note that this step is done by modifying the _clusters
     * field.
     * -# Repeat while any location changes its cluster number:
     *     - For each cluster, recalculate its centroid (update _centroids
     *       field).
     *     - Assign each location to a cluster (update _clusters field).
     *       Each location is assigned to the cluster that provides
     *       the shortest distance to its centroid.
     *
     * Modifier method
     */
    void run();

private:
    /**
     * Default random seed used to initialize the random number
     * generator used in the initialClusterAssignment() method.
     */
    static const unsigned int DEFAULT_RANDOM_SEED=1761560597U;

    /**
     * Vector of Location objects. This will be the input for the
     * clustering algorithm
     */
    VectorLocation _locations;

    /**
     * Number of clusters
     */
    int _K;

    /**
     * Vector of n integers (number of elements in _locations). This will
     * be the output of the clustering algorithm.
     * The integers in this vector define the cluster number (values in the
     * interval [0 - _K-1]) for each one of the Location object in _locations
     */
    VectorInt _clusters;
```

```
    /**
     * A vector of @p _K Location objects with the centroids of each cluster.
     */
    VectorLocation _centroids;

    /**
     * It indicates whether the run() method has already been executed to
     * obtain a clustering of the set of Location objects. The run() method
     * should set this field to true when it ends its execution.
     */
    bool _isDone;

    /**
     * Seed used to initialize the random number generator (with srand(seed)) in
     * the initialClusterAssignment() method.
     */
    unsigned int _seed;

    /**
     * This is the sum of the whithin-cluster variances of the clusters in
     * this Clustering object.
     * The value for this field should be set at the end of run() method
     * with the method calculateSumWCV(). Before running the
     * run() method, this field will likely contain a garbage value.
     * @note The value of the sum of the within-cluster variances can also be
     * retrieved using the method calculateSumWCV().
     * This field is introduced in this class to calculate its value once,
     * and thus avoiding having to recalculate it every time it
     * is needed
     */
    double _sumWCV;

    /**
     * This field represents the number of iterations used in the KMeans
     * algorithm until no cluster change occurred. Before running the
     * run() method, this field will likely contain a garbage value.
     * The value for this field should be set at the end of run() method.
     */
    int _numIterations;

    /**
     * @brief Performs an initial assignment of a cluster to each one of the
     * Location objects. This method begins by initializing the random number
     * generator using:
     * srand(seed);
     * Then, for each location i, assign cluster number rand() % _K to that
     * location.
     * Modifier method
     */
    void initialClusterAssignment();

    /**
     * @brief Calculates the sum of the within-cluster variances of this
     * Clustering object.
     * The within-cluster variance of a cluster C_j is calcultated with:
     * $\sum_{p_i \in C_j} squaredDistance(p_i, centroid_{C_j})$
     * @return A double with the sum of within-cluster variances of this
     * Clustering
     * Modifier method
     */
    double calculateSumWCV();
}; // end of class Clustering

#endif /* CLUSTERING_H */
```

7.5. ArrayClustering.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file ArrayClustering.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 22 de octubre de 2025, 11:57
 */

#ifndef ARRAYCLUSTERING_H
#define ARRAYCLUSTERING_H

#include "Clustering.h"

const int INITIAL_ARRAY_CLUSTERING_CAPACITY = 2;
const int ARRAY_CLUSTERING_CAPACITY_INCREMENT = 2;

/**
 * @struct ArrayClustering
 * @brief An ArrayClustering object contains a dynamic array of Clustering
 * objects, along with its capacity and current size.
 */
struct ArrayClustering{
    Clustering *clustering; // Pointer to the dynamic array of Clustering objects
    int capacity; // Capacity of the dynamic array
    int size; // Current number of Clustering objects stored in the array
};

/**
 * @brief Initializes the provided ArrayClustering with a given capacity. It
 * allocates a dynamic array of Clustering objects with the provided capacity,
 * sets the capacity field and initializes the size field to 0.
 * @throw std::out_of_range Throws a std::out_of_range exception if the
 * provided capacity is less than or equal to 0.
 * @param arrayClustering The ArrayClustering to initialize. Output parameter
 * @param capacity The initial capacity of the array. Input parameter
 */
void InitializeArrayClustering(ArrayClustering arrayClustering,
    int capacity=INITIAL_ARRAY_CLUSTERING_CAPACITY);

/**
 * @brief Deallocates the dynamic array of Clustering objects in the provided
 * ArrayClustering. It also sets the clustering pointer to nullptr, and the
 * capacity and size fields to 0.
 * @param arrayClustering The ArrayClustering to deallocate. Output parameter
 */
void DeallocateArrayClustering(ArrayClustering arrayClustering);

/**
 * @brief Searches for a given Clustering object in the provided
 * ArrayClustering. The search is performed by comparing each Clustering object
 * in the array with the provided Clustering object using the
 * Clustering::isEquivalentTo() method.
 * @param arrayClustering The ArrayClustering where to search will be
 * performed. Input parameter
 * @param clustering The Clustering object to search for. Input parameter
 * @return The position of the Clustering object in the array if found;
 * -1 otherwise.
 */
int FindArrayClustering(ArrayClustering arrayClustering,
                        Clustering clustering);

/**
 * @brief Appends the given Clustering object to the provided ArrayClustering
 * object. The Clustering object is only appended if the array does not
 * contain another Clustering object equivalent to the one being inserted.
 * If the array is full and the Clustering object must be appended to the array,
 * this function resizes the array with a capacity equal to the current
 * capacity plus an extra block of size equal to
 * ARRAY_CLUSTERING_CAPACITY_INCREMENT.
 * @param arrayClustering The ArrayClustering where the Clustering object will
 * be appended. Output parameter
 * @param clustering The Clustering object to append. Input parameter
 */
void AppendArrayClustering(ArrayClustering arrayClustering,
                           Clustering clustering);

/**
 * @brief Sorts the Clustering objects in the provided ArrayClustering in
 * ascending order based on their sum of within-cluster variances. If two
 * Clustering objects have the same sum of within-cluster variances, they are
 * further sorted by their number of iterations in ascending order.
 * @param arrayClustering The ArrayClustering to sort. Output parameter
 */
void SortArrayClustering(ArrayClustering arrayClustering);

#endif /* ARRAYCLUSTERING_H */
```

7.6. main.cpp

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file main.cpp
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 24 de octubre de 2025, 9:27
 */

#include <iostream>
#include <string>
#include <fstream>

#include "VectorLocation.h"
#include "Clustering.h"
#include "ArrayClustering.h"

using namespace std;

/**
 * Shows help about the use of this program in the given output stream
 * @param outputStream The output stream where the help will be shown (for example,
 * cout, cerr, etc)
 * @param message Additional message to show with the help
 */
void showHelp(std::ostream& outputStream, const string &message) {
    outputStream << "ERROR in Fraud2 parameters. " << message << endl;
    outputStream << "Run with the following arguments:" << endl;
    outputStream << "Fraud2 <K> <minSeed> <maxSeed> <inputFile.loc>" << endl;
    outputStream << endl;
    outputStream << "Parameters:" << endl;
    outputStream << "<K>: an integer with the number of clusters to use in the K-means algorithm" << endl;
    outputStream << "<minSeed>: an integer with the minimum seed value to use" << endl;
    outputStream << "<maxSeed>: an integer with the maximum seed value to use" << endl;
    outputStream << "<inputFile.loc>: name of the location file" << endl;
    outputStream << endl;
}

/**
 * The purpose of this program is to read a set of locations from
 * a loc file and make different clustering processes of these locations,
 * showing some statistics of the resulting clusterings on the standard output.
 *
 * The program first reads the locations from a file whose name is provided as
 * the last command line argument. It also reads from the command line an
 * integer K to set the number of clusters to be used in the clustering
 * processes. It also reads from the command line two integers minSeed and
 * maxSeed to set a range of seeds to be used in the different clustering
 * processes.
 * The program performs different clustering processes using the K-means
 * algorithm with seeds in the range from minSeed to maxSeed
 * (both included). The resulting Clustering objects are stored in an
 * ArrayClustering object, that represents a dynamic array of Clustering
 * objects. At the end, the program sorts the different Clustering objects
 * stored in the ArrayClustering object in ascending order of the sum of
 * within-cluster variances. If two Clustering objects have the same sum of
 * within-cluster variances, they are further sorted by their number of
 * iterations in ascending order.
 * Finally, the program shows some statistics about each of clustering stored
 * in the ArrayClustering object, including the value of K, the sum of the
 * within-cluster variances, and the number of iterations used by the
 * clustering algorithm until convergence.
 *
 * Be careful to show the output as in the below example.
 *
 * Running sintax:
 * > build/Fraud2 <K> <minSeed> <maxSeed> <inputFile.loc>
 *
 * Running example:
 * > build/Fraud2 5 0 3 ../Datasets/dataP2/princeton_names.loc
Clustering 0:
K=5
Sum of within-cluster variances: 194.455556
Number of iterations: 3

Clustering 1:
K=5
Sum of within-cluster variances: 233.319889
Number of iterations: 3

Clustering 2:
K=5
Sum of within-cluster variances: 250.044571
Number of iterations: 3
 */
int main(int argc, char* argv[]) {
    ArrayClustering arrayClustering; // Object to store the different clusterings
    VectorLocation locations; // Object to store the input locations
    Clustering clustering; // Clustering object
    int K; // Number of clusters to use
    unsigned int minSeed, maxSeed; // Range of seeds
    ifstream inputFile; // Input file stream for the loc file
```

```
    // Check if the number of arguments is valid.

    // Read K from the command line arguments

    // Read the seed range from the command line arguments
    // (use stoul to convert string to unsigned int)

    // Read from the input file the locations directly into the VectorLocation object

    // Initialize the arrayClustering object with an initial capacity of 2

    // For each seed in the given range, perform a clustering and store it in
    // arrayClustering


    // Sort the different Clustering objects stored in arrayClustering

    // Show statistics of each clustering in the sorted order

    // Deallocate the dynamic memory used by arrayClustering

    return 0;
}
```